Lecture 7
Unsupervised Learning — Clustering
AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas

ehirst@unicamp.br

Setembro 2026

Overview

1. Structure Without Labels
2. Centroid & Hierarchical Methods
3. Density & Spectral Methods
4. Clustering Geometric Data
5. Summary

Recap (Lecture 6): Transformers from attention weighting across tokens.
Guiding question: *How can you find structure in generic data?*

Unsupervised Clustering

The clustering task

Given points $\{x_i\}_{i=1}^N$ in a metric space (X, d) — *no labels*. Task: find structure.
Clustering asks for a partition $\{x_i\} = C_1 \sqcup \cdots \sqcup C_k$ into 'natural' groups.

- There is **no canonical definition** of a *cluster*:
  Compact and well-separated?
  Dense regions?
  Level sets of the sampling density?
  Connected components at a scale?
- Each algorithm *is* an implicit definition — the mathematical content of this lecture is which definition each algorithm encodes, and hence where it will succeed or fail.
- No test labels $\implies$ validation is subtler:
  Stability under resampling, comparison with known ground truth.

How is clustering useful in geometry?

Clustering detects:
$\rightarrow$ Regimes in parameter/moduli spaces.
$\rightarrow$ Phase-like families in databases of geometries.
$\rightarrow$ Connected components of real algebraic sets from samples.
$\rightarrow$ Orbits under discrete group actions.

k-Means: Voronoi Geometry

Objective

Choose centroids $\mu_1, \ldots, \mu_k$ minimising

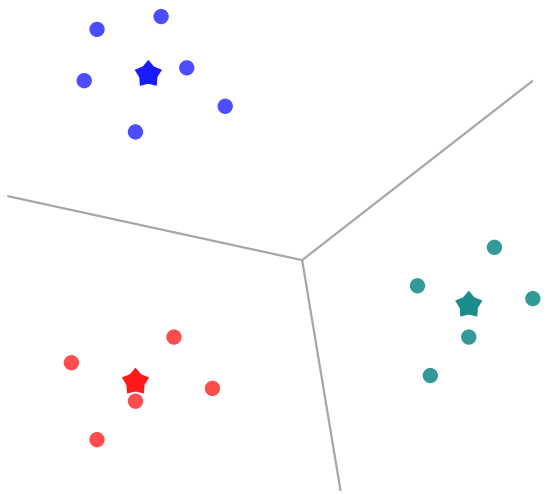
$$\sum_{i=1}^{N} \min_j \|x_i - \mu_j\|^2.$$

$\rightarrow$ NP-hard in general.
Lloyd's algorithm alternates:

1. Assign each x_i to nearest centroid ($\Rightarrow$ Voronoi cells);
2. Recompute each μ_j as the mean of its cell.

...iterating to convergence.

- Monotonically decreases the objective; converges to a *local* minimum — restart from several initialisations (k-means++).
- Implicit definition of cluster: **convex** (Voronoi cells are convex), isotropic, similar-size blobs.



Centroids (stars) induce a Voronoi partition (grey walls)
Lloyd's algorithm alternates assignment and averaging.

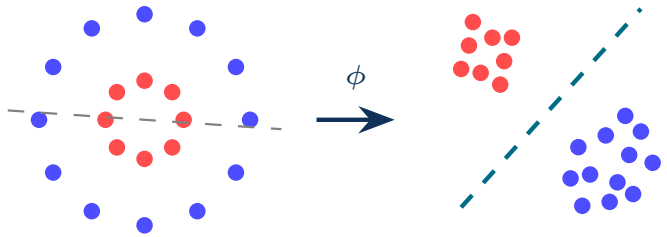
Diagnostics:

- **Elbow**: objective vs k; min $\implies$ best k.
- **Silhouette**:
  $s = \dfrac{\bar{d}_{\text{other}} - \bar{d}_{\text{own}}}{\max(\bar{d}_{\text{own}},\, \bar{d}_{\text{other}})} \approx 1 - \dfrac{\bar{d}_{\text{own}}}{\bar{d}_{\text{other}}}$,
  Mean distance to own vs. nearest other cluster; ≈ 1 good, ≈ 0 boundary, < 0 misassigned.
- **Stability**: structure survives resampling.

k-Means: Diagnostics and Kernels

Failure modes

→ **Non-convex clusters**: e.g. concentric circles.
→ **Unequal scales/densities**: a large diffuse cluster absorbs small tight one.
→ **Wrong** k: another number of clusters may be more natural $\longmapsto$ ELBOW method.



$\mathbb{R}^2$: no wall works $\mathcal{H}$: a wall works

Not convex, so k-means fails in $\mathbb{R}^2$; under ϕ the rings pull apart, the *convex* cells in $\mathcal{H}$ pull back to annuli.

Kernels (`sklearn` names):
→ `linear` $\langle x, y\rangle$ — plain k-means.
→ `poly` $(\gamma\langle x, y\rangle + r)^p$ — degree p.
→ `rbf` $e^{-\gamma\|x-y\|^2}$.
→ `precomputed` — your own K (e.g. G-invariant).

Kernel k-means: same algorithm, curved clusters

Embed $x \mapsto \phi(x) \in \mathcal{H}$ and run Lloyd *there*: cells convex in $\mathcal{H}$, preimages in $\mathbb{R}^n$ curved.
Kernel trick — ϕ is never evaluated. With $\mu_j = |C_j|^{-1} \sum_{y \in C_j} \phi(y)$,

$$\|\phi(x) - \mu_j\|^2 \;=\; k(x,x) \;-\; \tfrac{2}{|C_j|} \sum_{y \in C_j} k(x,y) \;+\; \tfrac{1}{|C_j|^2} \sum_{y,z \in C_j} k(y,z),$$

...so an assignment step needs only the Gram matrix $K_{ij} = k(x_i, x_j) = \langle \phi(x_i), \phi(x_j)\rangle_{\mathcal{H}}$:
→ Term one drops (same for all clusters).
→ $\mathcal{H}$ (infinite-dimensional for `rbf`) is never built.
→ Price: $O(N^2)$ per step, not $O(Nkn)$, and no centroid in $\mathbb{R}^n$.

Hierarchical Clustering: Multiscale Structure

Agglomerative algorithm

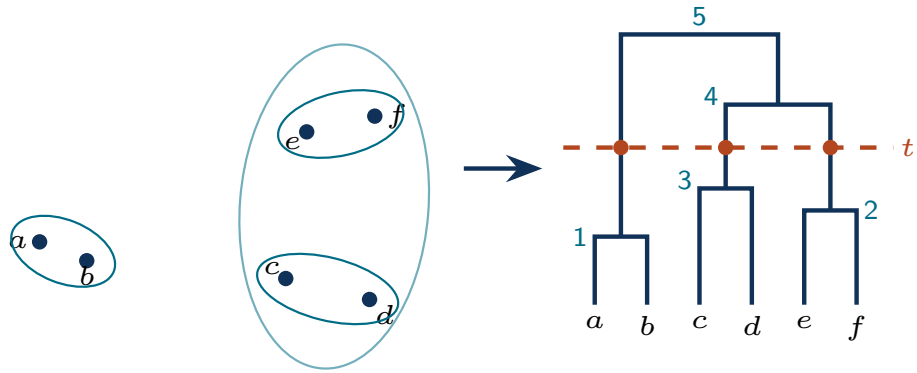
Bottom-up and greedy; k never fixed:

1. Start with N singletons.
2. Merge the closest two, at **merge height** $=$ their distance.
3. Repeat until one cluster is left.

$\rightarrow$ Performs $N-1$ merges, nested by construction.

Linkage $d(A, B)$, over $a \in A$, $b \in B$:

$\rightarrow$ `single`: $\min d(a,b)$ — chains.
$\rightarrow$ `complete`: $\max d(a,b)$ — blobs.
$\rightarrow$ `average`: $\text{mean}\, d(a,b)$ — neither.
$\rightarrow$ `ward`: least variance gain ($=$ greedy k-means).



Visualising the point merging: as nested groups, then as the tree they build.
Height $=$ distance merged at; the cut at t meets 3 stems $\implies$ scale t has 3 clusters.

- A *tall* gap between successive merges $\implies$ a stable clustering (here 3).
- The dendrogram graph at scale t $=$ Components of the t-neighbourhood graph $=$ the Min-Span-Tree with edges $> t$ cut.

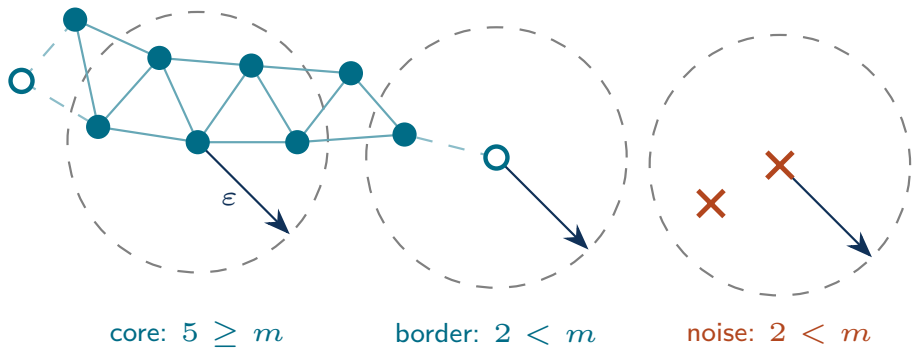
Geometer's reading

$\rightarrow$ The set of connected components $= \pi_0$ of the t-neighbourhood graph.
$\rightarrow$ Growing t is a filtration by scale, and pieces only merge, never split: the dendrogram *is* that merge tree, showing persistent H_0. Next: loops and voids (H_1, H_2).

DBSCAN: Clusters as Dense Regions

Core, border, noise

Fix a scale ε, a count m; every point is:
$\rightarrow$ **core**: $\geq m$ points within ε.
$\rightarrow$ **border**: fewer, but next to a core point.
$\rightarrow$ **noise**: neither.
Clusters = core-point components of the ε-graph; process stops at border points, noise is dropped.



$m = 4$ run with filled = core, hollow = border, $\times$ = noise. Each point has a disc of radius ε (not all are shown); teal edges are the ε-graph on core points.

- **DBSCAN = single linkage + a density filter**: drop the filter and it *is* single linkage at scale ε, the filter is exactly what stops chaining.
- Arbitrary shape, no k, and **noise points are excluded**.
- One global ε cannot track varying density; HDBSCAN sweeps ε and keeps whatever persists.

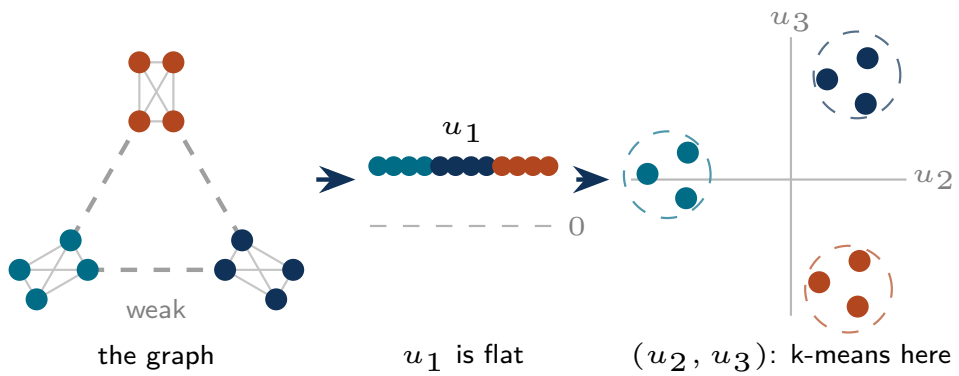
Where this goes: Persistent Homology

$\rightarrow$ Fill in the higher simplices and the ε-graph becomes the Vietoris–Rips complex.
$\rightarrow$ Persistent Homology sweeps ε to build a filtration of growing complexes, and tracks *homology* under the boundary operator $\longmapsto$ H_0 pieces, H_1 loops, H_2 voids.
$\rightarrow$ What survives a long range of ε are the real topological features (clustering only ever sees H_0, at one scale).

Spectral Clustering: Laplacian Eigenvectors

Algorithm

1. **Build a graph**: nodes = points, weights $w_{ij} = e^{-d(x_i,x_j)^2/2\sigma^2}$ (or kNN).
2. **Laplacian** $L = D - W$, $D_{ii} = \sum_j w_{ij}$; the graph's $-\Delta$: $(Lu)_i = \sum_j w_{ij}(u_i - u_j)$, value minus neighbour average. Take the k eigenvectors of smallest eigenvalue.
3. **Embed** $x_i \mapsto (u_1(i), \ldots, u_k(i)) \in \mathbb{R}^k$ and run k-means *there*.



u_1 is the *same* at every node ($L\mathbf{1} = 0$), so it separates nothing and is always dropped; $u_2, \ldots, u_k$ do the work.
Here $\lambda_2 = \lambda_3 \approx 0.55$ but $\lambda_4 = 4$ — the gap says $k = 3$.

- **Why**: $u^\top L u = \frac{1}{2}\sum_{ij} w_{ij}(u_i - u_j)^2$, the graph Dirichlet energy — low eigenvectors are the *smoothest* functions: near-constant inside a cluster, changing only across weak links.
- $L\mathbf{1} = 0$, so u_1 is constant. With c components $\lambda = 0$ has multiplicity c and $\ker L$ is spanned by their indicators; *nearly* separate clusters give near-indicators, hence tight blobs.
- u_2 (**Fiedler**): smoothest non-constant function, its sign a relaxed min ratio cut; Cheeger ties λ_2 to cut quality. Returns in Lectures 8, 10, 11.

The Metric Is the Model

Each clustering method is only as meaningful as its metric

$\rightarrow$ Each of the clustering algorithms considered uses distances.
$\rightarrow$ Choosing d is the actual modelling decision, and the step where *geometric expertise* enters.

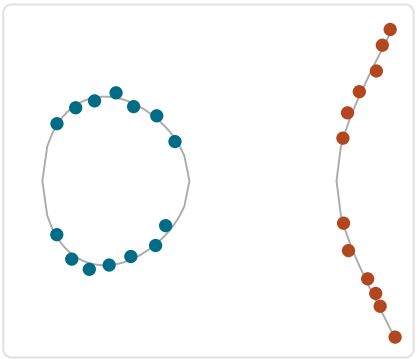
- **Feature scaling**: Coordinates in different units distort Euclidean distance; standardise, or choose weights deliberately.
- **Extrinsic vs intrinsic**: Two points close in $\mathbb{R}^n$ may be far on $\mathcal{M}$. Chordal distance in $\mathbb{R}^n$ vs geodesic distance along the underlying manifold — approximate the latter by shortest paths in the kNN graph (Isomap's trick, Lecture 8).
- **Invariant metrics**: If data has a symmetry group G (rotated point clouds, relabelled graphs), cluster in the quotient: $d([x],[y]) = \min_{g \in G} d(x, g \cdot y)$ — else clusters split into orbit fragments.
- **Non-Euclidean data**: When each data point is a shape, a measure, or a persistence diagram, use its own metric — Hausdorff and Gromov–Hausdorff for shapes (the latter ignoring position), Wasserstein for measures, bottleneck for diagrams. All four are a minimum over ways of matching one object onto the other.

Recommendation

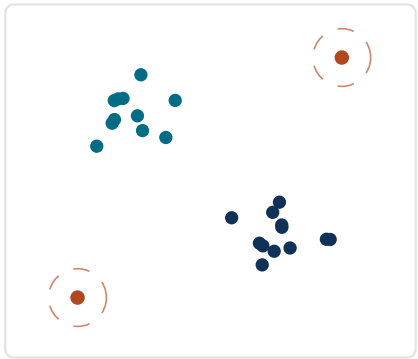
Before running any clustering algorithm, write down:
(i) What distance? (ii) On what representation? (iii) Invariant under what?

Applications in Geometry

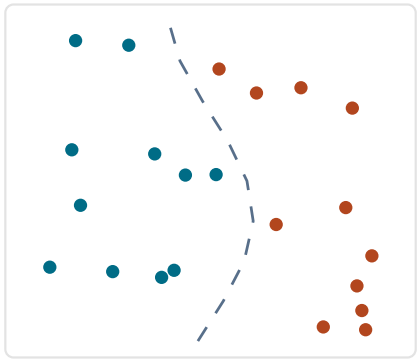
- **Components of real varieties**: Sample $\{f = 0\} \subset \mathbb{R}^n$,
  DBSCAN/spectral clustering recover connected components;
  ...numerical real algebraic geometry from samples.
- **Phases in databases**: Cluster feature vectors of large geometric datasets,
  (e.g. reflexive polytopes, graph ensembles) — clusters flag families/regimes
  ...for separate theoretical treatment; outliers flag exceptional objects (counter-examples?).
- **Moduli exploration**: Cluster numerically computed invariants across a parameter space to detect wall-crossing-like regime changes.
- **Symmetry detection**: Points of one orbit cluster under an invariant metric; the number of clusters estimates the number of orbits.



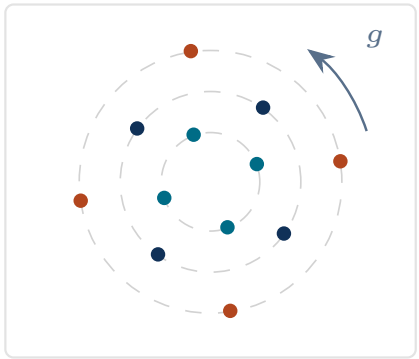
$\{f = 0\}$: two components



families + outliers



regimes across a wall



orbits $\Rightarrow$ clusters

Practice: the scikit-learn Workflow

Example (Three clusterers, one interface)

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN, SpectralClustering

Xs = StandardScaler().fit_transform(X)      # scale first!
labels_km = KMeans(n_clusters=3, n_init=10).fit_predict(Xs)
labels_db = DBSCAN(eps=0.3, min_samples=8).fit_predict(Xs)
labels_sp = SpectralClustering(n_clusters=3,
              affinity="nearest_neighbors").fit_predict(Xs)
```

- **Complexity** (N points, k clusters, n dimensions):

	k-means	kernel k-means	DBSCAN	hierarchical	spectral
compute	$O(Nkn)$ / iter.	$O(N^2)$ / iter.	$O(N \log N)$	$O(N^2 \log N)$	$O(N^2)$–$O(N^3)$
memory	$O(Nn)$	$O(N^2)$	$O(N)$	$O(N^2)$	$O(N^2)$

  Only k-means is linear in N — a dense $N \times N$ matrix (Gram or distance) at $N = 10^4$ is $\sim$800 MB, so subsample or sparsify (kNN graph: $O(N)$).

- **Evaluation**: without labels there is no accuracy — ask three questions instead.
  - *Stability*: re-run on bootstrap resamples, new seeds, perturbed ε or k; keep only structure that survives — and use it to choose k.
  - *Internal* (geometry alone): silhouette score. **Caveat**: these reward compact convex blobs, quietly favouring k-means over the shapes DBSCAN and spectral exist for.
  - *External* (ground truth known — synthetic data, or geometry you can compute): adjusted Rand index, chance-corrected so a random labelling scores 0 and an exact match 1.
- Always *visualise* (via Lecture 8's dimensionality reduction) before believing any clustering.

Summary

Key points

Clustering has *no canonical definition*: each algorithm encodes one...

Convex Voronoi blobs (k-means), Multiscale merges (hierarchical),
Dense connected regions (DBSCAN), Graph connectivity (spectral).

- **k-means**: linear in N and simple — but you fix k, and anisotropy breaks it.
- **Hierarchical**: one dendrogram, every scale at once — the linkage is the model.
- **DBSCAN**: any shape, plus explicit noise — H_0 of a Rips filtration.
- **Spectral**: a Laplacian eigenproblem — the course's first spectral geometry.

The metric is the model: scale features, respect symmetry, choose intrinsic distances when the manifold matters.

Next

Tutorial 7: Comparing methods; k-means vs DBSCAN vs spectral on manifold samples.
Lecture 8: Dimensionality reduction — seeing high-dimensional geometric data.